

Oracle Lab in Minutes

Manuscript for DOAG 2025 Container Hands-On Labs

November 20, 2025, Version 0.1.0



Table of Contents

1	Oracle Lab in Minutes — Manuscript	3
1.1	<i>TDE, Standby, Unified Audit & More with Docker Containers</i>	3
1.2	Why Fast & Portable Oracle Labs?	3
1.3	Docker-Based Oracle Free Environments	4
1.4	Automating Everything at Startup	5
1.5	Live Demos	6
1.6	Best Practices & Use Cases	7
1.7	Building Your Own Oracle Containers	7
1.8	Summary & Q&A	8
1.9	References	8

List of Figures

1 Oracle Free Labs 6

1 Oracle Lab in Minutes — Manuscript

1.1 TDE, Standby, Unified Audit & More with Docker Containers

Author: Stefan Oehrli **Accenture - Modern Data Platforms Oracle ACE Director**

1.2 Why Fast & Portable Oracle Labs?

1.2.1 Friday Afternoon and Time to Test...?

Many Oracle features deserve proper exploration, yet suitable test environments are often missing. Key challenges include selecting the right Oracle release and RU level, ensuring reproducible setups, validating bug fixes, and troubleshooting issues reliably. Testing new features or confirming patch behavior is difficult if the environment is inconsistent or time-consuming to prepare.

1.2.2 What Can We Test in a Local Oracle Lab?

A local container-based lab allows fast experimentation across a wide range of topics, including the full spectrum of your Maximal Database Security Architecture. This includes encryption, auditing, least privilege, secure configuration, multitenant operations, and more—without relying on large or shared infrastructure.

1.2.3 “But We Do Have a Test System...”

Existing test systems often fall short because they are:

- already used by someone else
- under maintenance or patching
- outdated or inconsistent
- misconfigured for specific tests
- not scalable for multiple users
- limited in system resources
- tied to licensing constraints

Time for engineering and testing is limited. It should not be spent preparing or repairing lab systems.

1.2.4 Solution Approaches

Traditional options include on-premises servers, local VMs, cloud engineering labs, or automation-heavy setups using tools like Ansible or Puppet. However, these approaches involve high effort, cost, limited availability, or lack offline capability.

A modern alternative is a **portable Docker-based Oracle lab**: fast, reproducible, lightweight, and available anywhere—even offline.

1.2.5 Limitations of Local VM-Based Labs

VMs require significant CPU and memory, consume large storage volumes due to snapshots, and are not ideal for frequent resets or parallel test cycles. The setup effort alone often outweighs the actual testing time.

1.2.6 Requirements for an Ideal Lab Setup

An effective test environment should offer:

- fast provisioning and cleanup
- low resource usage
- repeatability and portability
- quick access to advanced Oracle features

1.3 Docker-Based Oracle Free Environments

1.3.1 Why Containers for Oracle?

Containers start in seconds, use minimal system resources, and are easy to reset or clone. They provide isolated, reproducible environments ideal for iterative testing or training sessions.

1.3.2 Docker Components

When using Docker/Podman, we work with:

- **Images:** immutable definitions
- **Containers:** runtime instances of images
- **Volumes:** external storage for persistence
- **Networks:** isolated communication layers

1.3.3 Ensuring Data Persistence — Volumes & Bind Mounts

Docker images are immutable and container filesystem changes disappear once a container is deleted. Persistence requires **volumes** or **bind mounts**, ensuring all Oracle database files live outside the container. Lifecycle remains consistent: new image => new container => same volume.

1.3.4 Oracle Free Edition as a Container

Oracle Free runs reliably on Docker and Podman, offering a meaningful subset of Oracle Database features—perfect for labs, training sessions, and PoCs. Configuration is straightforward with `.env` files and Compose profiles. Resources include:

- Oracle Container Registry
- Oracle Database Repositories
- Oracle Docker GitHub projects
- Docker Hub Oracle Free images
- OraDBA GitHub repositories

1.3.5 Platform Support: Intel & ARM

The same images and scripts run identically on Intel and ARM platforms, including macOS with Apple Silicon (M1/M2/M3). This makes Oracle labs highly portable and ideal for laptop-based engineering.

1.4 Automating Everything at Startup

1.4.1 Automation at Container Startup

Initialization scripts automatically configure the database on container start—no manual steps required. This ensures consistent setups ideal for workshops, demos, and engineering tasks.

1.4.2 What Can Be Automated?

Automation covers:

- PDB creation and configuration
- TDE setup and keystore management
- Unified Audit policies
- Schema and user creation
- Lightweight standby/replication setups
- Loading demo data
- Executing SQL or shell scripts

1.4.3 Using Prebuilt PDB Archives

PDB archives provide instant access to complete, preconfigured database environments. This eliminates installation time and ensures reproducibility—ideal for trainings or complex demos.

1.4.4 Building Complex Scenarios Automatically

Compose profiles can orchestrate multiple services, sharing scripts and enabling multi-PDB or multi-instance labs. This supports production-like PoC setups involving TDE, Unified Audit, Standby concepts, or repository-based environments.

1.5 Live Demos

1.5.1 Live Demo Overview

The demonstration covers:

- starting Oracle in seconds
- enabling TDE automatically
- configuring Unified Audit
- exploring PDB operations
- preparing a lightweight standby setup



Figure 1: Oracle Free Labs

1.5.2 Demo Flow

1. Start the lab environment
2. Check readiness via container logs
3. Inspect the base configuration
4. Apply automated features

Startup the *odbenc* container:

```
cd oracle-free-labs
docker compose --profile odbenc up -d
```

Check the logfiles:

```
docker compose --profile odbenc logs -f
```

Wait until you see

```
#####
DATABASE IS READY TO USE!
#####
```

Access container via Shell:

```
docker exec -it odbenc bash -l
```

1.6 Best Practices & Use Cases

1.6.1 Typical Use Cases

- Security feature testing
- Application regression
- Architecture validation
- Training and onboarding
- Troubleshooting
- Proof-of-Concept environments

1.6.2 Best Practices

- Keep configuration centralized and versioned
- Use `.env` for consistency
- Separate data, scripts, and scenarios
- Prefer init scripts over manual steps
- Reset environments regularly
- Document all customizations
- Store labs directly in Git where feasible

1.6.3 Working with Profiles & `.env` Files

Compose profiles define scenario sets. Environment variables control core parameters such as passwords, ports, and DB settings. This enables simple switching between environments and ensures reproducibility across platforms.

1.7 Building Your Own Oracle Containers

1.7.1 Options for Building Oracle Containers

- Use Oracle's official container build scripts
- Extend the base images via init scripts
- Build fully custom images when needed

Your own ongoing work includes:

- automated Database container builds via AutoUpgrade
- automatic RU downloads
- multi-platform builds
- new GitHub project: **oehrlis/oracle-database-docker**

1.7.2 Key Considerations for Custom Images

- Patch level management (RU, OJVM, OPatch)
- Intel vs ARM multi-platform builds
- No passwords or secrets inside images
- Avoid embedding wallets/configuration
- Track base image versions carefully
- Understand relevant licensing requirements

1.7.3 Structure & Maintainability

- Modular script and config layout
- Use shared folders for common logic
- Separate build-time and runtime logic
- Keep automation in version control
- Document image layers and customizations

1.8 Summary & Q&A

1.8.1 Summary

Container-based Oracle labs provide:

- Oracle environments in minutes
- Lightweight, portable labs for Intel & ARM
- Fully automated and reproducible setups
- Ideal tooling for training, engineering, and testing
- Ability to model complex scenarios without complex infrastructure

1.8.2 Slogan

Build your Oracle labs with proven scenarios, efficient automation, and embrace containers for fast, portable, and reproducible database environments.

1.8.3 Q&A

Open for questions, discussions, and sharing additional examples from the field.

1.9 References

1.9.1 Oracle Resources

- Oracle Container [Registry](#)

- Oracle Database [Repositories](#)
- Oracle Docker Images on [GitHub](#)
- Oracle Free on Docker Hub by [Gerald Venzl](#)

1.9.2 My Community Content

- **OraDBA Blog:** <https://www.oradba.ch> and specific the [Docker tag](#).
- Legacy Docker repo: [oehrli/docker](#)
- Oracle Free Labs: [oehrli/oracle-free-labs](#)
- New Docker repo: [oehrli/oracle-database-docker](#)